

Discovery: Integração Wix → Firebase (projeto-ovo)

Contexto

O site da Ovo no Wix já concentra dados reais de negócio — alunos/contatos, cursos/turmas, matrículas e pagamentos das compras feitas por lá. Hoje o Trilho (projeto-ovo) só tem esses dados quando alguém digita manualmente ou faz o import CSV único em `/pessoas/importar`. O objetivo de longo prazo é que Pessoas, Turmas e Caixa reflitam automaticamente o que já acontece no Wix, evitando redigitação e dado desatualizado.

Este documento é só **discovery** — mapeia o que existe hoje no projeto, o que a API do Wix oferece, e o que falta descobrir/decidir antes de qualquer implementação. Nenhum código foi alterado a partir dele.

O que já existe no projeto-ovo

- **Modelo de dados atual não tem identificador externo.** `Pessoa`, `Turma` e `Matricula` (`src/core/pessoas/schema.ts`, `src/core/turmas/schema.ts`, `src/core/matriculas/schema.ts`) não têm campo tipo `wixContactId` / `wixOrderId`. Sem isso, não dá para fazer sync idempotente (só criar/atualizar, nunca duplicar) — seria necessário adicionar esses campos antes de uma integração contínua.
- **Recebimento e Repasse já têm o campo origem: "wix" | "manual"** (`src/core/financeiro/shared.ts`) — é o gancho de produto já pensado para dado importado do Wix aparecer distinguível na tela de Caixa. `Pessoa` / `Turma` / `Matricula` não têm esse campo ainda.
- **Dedup hoje é só por nome normalizado** (import CSV em `src/app/(protected)/pessoas/importar/actions.ts`) — é alerta visual, não bloqueio, e não serve para sync recorrente.
- **Toda escrita no Firestore passa pelo Admin SDK server-side** via `getFirebaseAdminFirestore()` (`src/core/firebase/firebaseAdmin.ts`). Uma futura rota de sync (webhook ou cron) seguiria o mesmo padrão — Server Action ou route handler em `src/app/api/`, sem tocar em `firestore.rules`.
- **Isso já está no roadmap do produto**, categorizado como v4 (deliberadamente depois de v1-v3, ver `docs/mini-prd.md`):
 1. Prioridade 1: import de **recebimentos** do Wix/Pagar.me ("maior ganho de tempo da Camila").
 2. Prioridade 2: import da **base de alunos**.

- **Regra de produto já fechada** (docs/context.md): "O Trilho registra e reflete; nunca executa a ação externa" — ou seja, a integração deve ser **somente leitura** do Wix para o Firebase, nunca escrever de volta no Wix.
- **Nenhuma dependência de Wix, webhook ou cron existe hoje** no projeto. `fetch` nativo do Next.js já cobre chamadas HTTP; não há SDK do Wix instalado.

Panorama da API do Wix

Autenticação — API Key é o caminho certo aqui

O Wix tem dois modelos de autenticação:

- **OAuth** — obrigatório para apps de terceiro publicados no App Market (não é o nosso caso).
- **API Key** — gerada por um account owner/co-owner em manage.wix.com/account/api-keys, pensada exatamente para o nosso caso de uso: "integrações externas" e "self-managed headless projects" acessando os próprios dados da conta.

Detalhes relevantes:

- A chave pode ser restrita a **sites específicos** (recomendado: restringir só ao site da Ovo) e a um **conjunto de permissões** escolhido na criação.
- Chamadas em nível de site precisam da API key + do `site ID` (obtido via `Query Sites` ou na URL do dashboard do site).
- **Bloqueio atual:** ninguém do lado do projeto tem acesso de owner/co-owner na conta Wix ainda — é preciso pedir para quem tem (ex.: Camila) gerar a chave quando formos implementar.

Qual API é a fonte dos "cursos/turmas" — ainda não sabemos

Ainda não se sabe qual produto Wix a escola usa para vender os cursos no site. Isso muda qual API é a fonte primária de dados, então mapeamos as três possibilidades mais prováveis:

Se a Ovo usa...	API relevante	O que ela expõe
Wix Bookings (cursos/aulas como serviços agendáveis)	Bookings Reader V2 + Attendance API	Serviços (cursos), sessões, booking por cliente, frequência por sessão
Wix Pricing Plans (mensalidade recorrente / assinatura)	Pricing Plans API (Plans + Orders)	Planos (ex. "Mensalidade Turma X"), orders com status de assinatura (ativa/pausada/cancelada) — modelo <code>subscription</code>

Se a Ovo usa...	API relevante	O que ela expõe
Wix Stores (curso vendido como produto avulso)	Stores Catalog API (V1 ou V3) + eCommerce Orders API	Produtos no catálogo, pedidos com itens/preço/status de pagamento

É bem possível que seja uma **combinação** (ex.: Pricing Plans para mensalidade + Bookings para controlar turma/frequência). Isso só se resolve olhando o painel real do site.

Para os **alunos/contatos**, independente do produto acima:

- **Contacts API** (`crm/members-contacts/contacts`) — lista até 1000 contatos por chamada (`List Contacts`), com `Query Contacts` para filtros. Provavelmente a fonte principal de "pessoa" (nome, e-mail, telefone).
- **Members API** — só relevante se a Ovo usa a Área de Membros do site (login de aluno no site); nem toda loja Wix tem isso ativado.

Para **eventos avulsos** (se a escola também vende workshops/eventos pontuais, não só turmas regulares): **Wix Events API** (Tickets + Event Guests) é outra fonte possível, separada de Bookings/Stores.

Webhooks vs. polling — os dois são viáveis

- **Webhooks:** o Wix envia POST (payload em JWT, assinado — dá pra verificar autenticidade) para uma URL nossa quando um evento acontece (ex. novo pedido, contato atualizado). Configurado no painel do app/site Wix. É a opção "tempo real", mas exige um endpoint público (`src/app/api/.../route.ts`) sempre no ar.
- **Polling/cron:** chamar `Query Contacts` / `Search Orders` / etc periodicamente (ex. a cada X horas) e comparar com o que já está no Firestore. Mais simples de operar, sem depender de endpoint público sempre disponível, mas não é tempo real e precisa de lógica de "desde quando" (cursor por data de atualização, se a API suportar).

Dado que o produto já tem o hábito de "conferir o Caixa periodicamente" (não é um app real-time), **polling/cron parece o ponto de partida mais simples e alinhado ao que já existe** — mas isso é uma recomendação a validar, não uma decisão fechada.

Lacunas e riscos identificados

1. **Sem identificador externo no schema.** Antes de qualquer sync automático, `Pessoa` , `Turma` , `Matricula` (e talvez `Recebimento` / `Repasse`) precisariam ganhar um campo como `wixContactId` / `wixOrderId` / `wixPlanId` para permitir upsert idempotente (não duplicar a cada sync).

2. **Ainda não sabemos qual produto Wix a escola usa para vender curso** — bloqueia decidir a API exata (Bookings vs Pricing Plans vs Stores). Precisa ser verificado no painel Wix da Ovo.
3. **Ninguém do lado do projeto tem acesso de owner/co-owner na conta Wix** — bloqueia gerar a API key. Precisa pedir para quem tem (ex. Camila).
4. **Modelo Pessoa atual não distingue aluno menor de responsável financeiro**, nem tem e-mail/telefone/documento — o Wix provavelmente traz esses dados (contato tem e-mail/telefone nativamente); pode ser a oportunidade de enriquecer o schema, mas é uma decisão de produto, não só técnica.
5. **Mapeamento de "turma" Wix → "turma" Trilho não é 1:1 garantido**. Ex.: se for Pricing Plans, um "plano" pode não corresponder exatamente a uma Turma do Trilho (que tem mensalidade, repasse, educador). Precisa decisão de produto sobre como cada conceito Wix vira um conceito Trilho.
6. **Rate limits e paginação** existem em todos os endpoints (ex. Contacts: até 1000/chamada; Tickets: até 100/chamada) — relevante para desenhar o cron/import em lotes, mas não é bloqueio, só detalhe de implementação futura.

Próximos passos recomendados (ainda discovery, não implementação)

1. **Verificar no painel do Wix da Ovo** qual app vende os cursos — Bookings, Pricing Plans, Stores, ou combinação — e quais dados de pagamento aparecem lá (Wix Payments nativo? Pagar.me integrado, como os docs já mencionam?).
2. **Pedir para o owner/co-owner da conta Wix** (provavelmente Camila) gerar uma API key restrita ao site da Ovo, e confirmar o site ID, só quando formos para a fase de implementação (não precisa agora).
3. Com a resposta do passo 1, revisitar este discovery para fechar: qual(is) API(s) exatas usar, e desenhar o mapeamento de campos Wix → schema Trilho (Pessoa , Turma , Matricula , Recebimento).
4. Decidir, como time, a ordem de escopo — os docs já sugerem começar por **recebimentos** (mais valor, menor mudança de schema) antes da **base de alunos** (mudança maior, precisa de campo de identificador externo em Pessoa).
5. Só depois disso desenhar a decisão técnica webhook vs. cron e o formato exato da rota/Server Action de sync.

Referências

- [About the Contacts API](#)
- [About the eCommerce Orders API](#)

- [About the Bookings APIs](#)
- [About the Wix Events API](#)
- [About the Wix Stores Catalog API](#)
- [About the Pricing Plans APIs](#)
- [About API Keys](#)
- [About Webhooks](#)